

Gray To Binary Encoder + Binary To Seven Segment Implementation

Name: Youssef Hossam Eldeen Mohamed Elzainy Mostafa Elzainy

ID: 212600

Gray To Binary Encoding

Behavioral Level

```
module GrayToBinary_beh #(
    parameter WIDTH = 4
) (
    input [WIDTH - 1 : 0] gray,
    output reg [WIDTH - 1 : 0] binary
);

    integer i;

    always @(*) begin
        binary[WIDTH-1] = gray[WIDTH-1];

        for (i = WIDTH-2; i >= 0; i = i-1) begin
            binary[i] = binary[i+1] ^ gray[i];
        end
    end

endmodule
```

Gate Level

```
module GrayToBinary_gate #(
    parameter WIDTH = 4
) (
    input [WIDTH - 1 : 0] gray,
    output [WIDTH - 1 : 0] binary
);

    assign binary[WIDTH-1] = gray[WIDTH-1];

    genvar i;
```

```

generate
    for (i = WIDTH-2; i >= 0; i = i-1) begin

        xor(binary[i], binary[i+1], gray[i]);

    end
endgenerate

endmodule

```

Testbench

```

module GrayToBinary_tb;
    parameter WIDTH = 6;
    reg [WIDTH - 1: 0] gray;
    wire [WIDTH - 1: 0] binary_beh, binary_gate;
    integer i;

    GrayToBinary_beh #(
        .WIDTH(WIDTH)
    ) dut_beh (
        .gray(gray),
        .binary(binary_beh)
    );

    GrayToBinary_gate #(
        .WIDTH(WIDTH)
    ) dut_gate (
        .gray(gray),
        .binary(binary_gate)
    );

    initial begin
        for (i = 0; i < 64 ; i = i + 1) begin
            gray = i;
            #10;

            if (binary_beh != binary_gate) begin
                $display("Error - values not equal");
                $stop;
            end
        end
    end
end

```

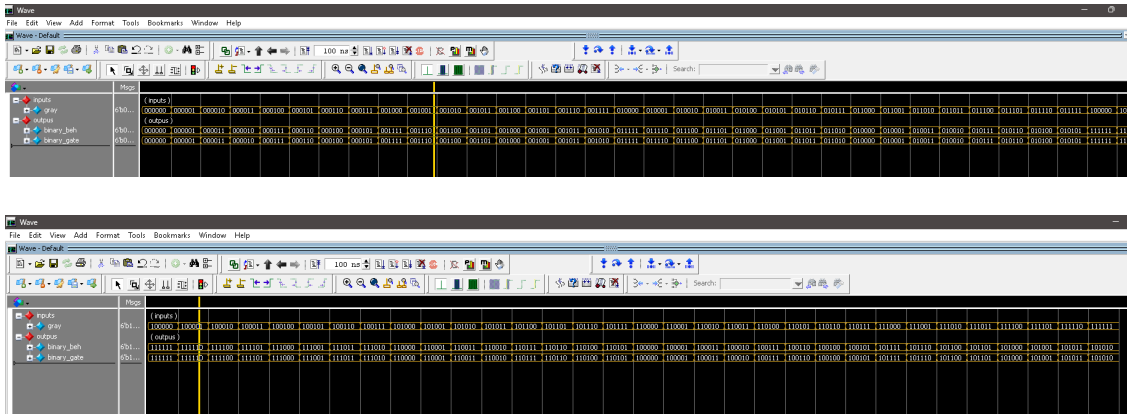
```

$finish;

end
endmodule

```

Waveform



Binary To Seven Segment Decoding

Design

```

module BinToSeg_Decoder (
    input [3:0] Bin_in,
    output reg [6:0] Seg_out
);
    localparam [6:0]
        SEG_0 = 7'b0000001,
        SEG_1 = 7'b1001111,
        SEG_2 = 7'b0010010,
        SEG_3 = 7'b0000110,
        SEG_4 = 7'b1001100,
        SEG_5 = 7'b0100100,
        SEG_6 = 7'b0100000,
        SEG_7 = 7'b0001111,
        SEG_8 = 7'b0000000,
        SEG_9 = 7'b0000100,
        SEG_A = 7'b0001000,
        SEG_B = 7'b1100000,
        SEG_C = 7'b0110001,
        SEG_D = 7'b1000010,
        SEG_E = 7'b0110000,
        SEG_F = 7'b0111000;

```

```

always @(*) begin
    case (Bin_in)
        4'b0000: Seg_out = SEG_0;
        4'b0001: Seg_out = SEG_1;
        4'b0010: Seg_out = SEG_2;
        4'b0011: Seg_out = SEG_3;
        4'b0100: Seg_out = SEG_4;
        4'b0101: Seg_out = SEG_5;
        4'b0110: Seg_out = SEG_6;
        4'b0111: Seg_out = SEG_7;
        4'b1000: Seg_out = SEG_8;
        4'b1001: Seg_out = SEG_9;
        4'b1010: Seg_out = SEG_A;
        4'b1011: Seg_out = SEG_B;
        4'b1100: Seg_out = SEG_C;
        4'b1101: Seg_out = SEG_D;
        4'b1110: Seg_out = SEG_E;
        4'b1111: Seg_out = SEG_F;

        default: Seg_out = 7'b1111111;
    endcase
end

endmodule

```

Testbench

```

module BinToSeg_Decoder_tb;
    reg [3:0] Bin_in;
    wire [6:0] Seg_out;
    integer i;

    BinToSeg_Decoder dut (
        .Bin_in(Bin_in),
        .Seg_out(Seg_out)
    );

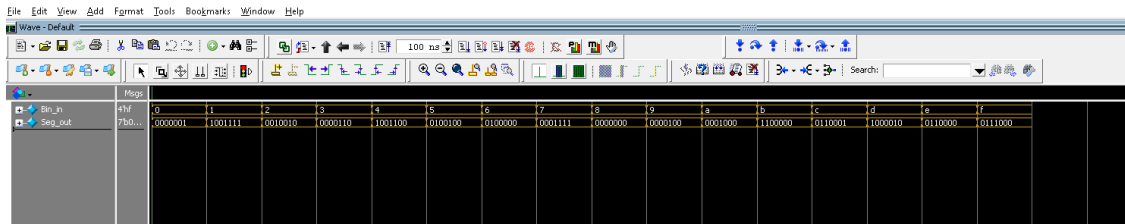
    initial begin
        for (i = 0; i < 16 ; i = i + 1 ) begin
            Bin_in = i;

            #10;
        end
        $finish;
    end
end

```

```
endmodule
```

Waveform



Encoder + Decoder Implementation

Top module

```
module GrayToSeg (  
    input [3:0] Gray_in,  
    output [6:0] Seg_out  
);  
    wire [3:0] bin_out_in;  
    GrayToBinary_gate dut_gray (  
        .gray(Gray_in),  
        .binary(bin_out_in)  
    );  
  
    BinToSeg_Decoder dutseg (  
        .Bin_in(bin_out_in),  
        .Seg_out(Seg_out)  
    );  
  
endmodule
```

Testbench

```
module GrayToSeg_Decoder_tb;  
    reg [3:0] gray_in;  
    wire [6:0] Seg_out;  
    integer i;  
  
    GrayToSeg dut (  
        .Gray_in(gray_in),  
        .Seg_out(Seg_out)  
    );  
endmodule
```

```

);

initial begin
    for (i = 0; i < 16 ; i = i + 1 ) begin
        gray_in = i;

        #10;
    end
    $finish;
end

endmodule

```

Waveform

